The measuredtex package*

Blair Hall
Measurement Standards Laboratory of New Zealand

2026-06-02

Abstract

The measuredtex package provides semantic markup for measurement-model documents. It supplies environments for typesetting model equations while writing structured model metadata to a sidecar file, commands for recording definitions of mathematical terms, and a lightweight mechanism for collecting textual concepts used in the document. The package currently writes model and mathematical-term records to `\jobname.models`. Textual concept records collected with `\mdconcept` are written at the end of the document to `\jobname.terms`.

Contents

*This document corresponds to measuredtex v0.1, dated 2026-06-02.

1 Introduction

Measurement-model documents often contain mathematical expressions, definitions of the symbols used in those expressions, and ordinary textual terminology that should be available for downstream processing. The measuredtex package supports this by coupling ordinary LaTeX markup with simple semantic annotations.

The package is deliberately modest. It does not attempt to define a full model-interchange language. Instead, it records selected information from the source document in plain-text sidecar files during compilation. These sidecar files can then be inspected directly or processed by external tooling.

The package provides three related facilities:

- model environments that wrap standard `amsmath` display environments and write model metadata;
- commands and environments for recording definitions of mathematical terms associated with a labelled model equation;
- a command for marking textual concepts and writing a separate concept list.

2 Usage

Load the package in the usual way:

```
\usepackage{measuredtex}
```

2.1 Model environments

mdeq

```
\begin{mdeq}[⟨metadata⟩]
⟨equation body⟩
\end{mdeq}
```

Typesets ⟨*equation body*⟩ in a standard `equation` environment and writes a model record to `\jobname.models`.

The optional ⟨*metadata*⟩ argument is a key–value list. The recognised keys are:

`name` a human-readable name for the model;

`role` the role of the equation in the document;

`source` a source or provenance string;

`label` the LaTeX label to apply to the displayed equation.

If the `label` key is supplied, the same value is written to the sidecar file and applied as a LaTeX label inside the equation environment.

For example:

```
\begin{mdeq}[name=Ohm's law, role=model, label=eq:ohm]
  V = IR
\end{mdeq}
```

The body of the environment is typeset as an equation. A model record is written to `\jobname.models`.

`mdalign` `\begin{mdalign}[`⟨*metadata*⟩`]`
⟨*aligned equation body*⟩
`\end{mdalign}`

The `mdalign` environment typesets ⟨*aligned equation body*⟩ in a standard `align` environment and writes a model record to `\jobname.models`.

The optional ⟨*metadata*⟩ argument is a key–value list. The recognised keys are `name`, `role`, `source`, and `label`.

If the `label` key is supplied, the same value is written to the sidecar file and applied as a LaTeX label inside the displayed `align` environment.

For example:

```
\begin{mdalign}[name=Linear model, role=model, label=eq:linear]
  y &= ax + b \\
  u_y^2 &= x^2 u_a^2 + a^2 u_x^2 + u_b^2
\end{mdalign}
```

2.2 Mathematical term definitions

Mathematical terms may be associated with a model equation by referring to the equation label. Term records are written to `\jobname.models`. The package checks whether the supplied equation label has been defined and issues a warning if it has not.

`\mddefntext` `\mddefntext{`⟨*label*⟩`}{`⟨*term*⟩`}[`⟨*separator*⟩`]`
`{`⟨*definition*⟩`}`

Writes a mathematical-term definition record to `\jobname.models` and also typesets an inline textual definition.

The ⟨*label*⟩ argument is the LaTeX label of the model equation to which the term belongs. The ⟨*term*⟩ argument is typeset in inline mathematics mode. The optional ⟨*separator*⟩ argument is rendered between the term and the definition text. The separator is not written to the sidecar file. The ⟨*definition*⟩ argument is rendered and written to the sidecar file.

If ⟨*label*⟩ has not been defined, the package issues an `undefined-label` warning.

For example:

```
\mddefntext{eq:ohm}{V}[:]{the voltage across the resistor}
```

`\mddefntab` `\mddefntab{⟨label⟩}{⟨term⟩}{⟨definition⟩}`

Writes a mathematical-term definition record to `\jobname.models` and typesets a two-column table-row fragment.

The ⟨*label*⟩ argument is the LaTeX label of the model equation to which the term belongs. The ⟨*term*⟩ argument is typeset in inline mathematics mode. The rendered output consists of the term, followed by an alignment character, followed by ⟨*definition*⟩.

This command is intended for use inside table-like environments such as `tabular` or `tabularx`. It does not terminate the row; the surrounding table markup should provide any required `\\`.

If ⟨*label*⟩ has not been defined, the package issues an `undefined-label` warning.

For example:

```
\begin{tabular}{ll}
  \mddefntab{eq:ohm}{V}{The voltage across the resistor} \\
  \mddefntab{eq:ohm}{I}{The current through the resistor} \\
  \mddefntab{eq:ohm}{R}{The resistance} \\
\end{tabular}
```

`mddescription` `\begin{mddescription}`
`\mditem` `\mditem{⟨label⟩}{⟨term⟩}{⟨definition⟩}`
`...`
`\end{mddescription}`

The `mddescription` environment provides a description-list form for mathematical term definitions. It is a wrapper around the standard `description` environment.

Within this environment, `\mditem{`⟨*label*⟩`}{`⟨*term*⟩`} {`⟨*definition*⟩`}` writes a mathematical-term definition record to `\jobname.models` and typesets a description-list item. The item label is ⟨*term*⟩, typeset in inline mathematics mode, and the item body is ⟨*definition*⟩.

The ⟨*label*⟩ argument is the LaTeX label of the model equation to which the term belongs. If ⟨*label*⟩ has not been defined, the package issues an `undefined-label` warning.

For example:

```
\begin{mddescription}
  \mditem{eq:ohm}{V}{The voltage across the resistor}
  \mditem{eq:ohm}{I}{The current through the resistor}
  \mditem{eq:ohm}{R}{The resistance}
\end{mddescription}
```

\mddefn `\mddefn{⟨label⟩}{⟨term⟩}{⟨definition⟩}`

Writes a mathematical-term definition record to `\jobname.models`. The ⟨*label*⟩ argument is the LaTeX label of the model equation to which the term belongs. The ⟨*term*⟩ argument is the mathematical term being defined, and ⟨*definition*⟩ is the corresponding definition text.

This command does not typeset any visible material in the document. It is intended for cases where the term definition should be recorded semantically without being rendered at the point where the command appears.

If ⟨*label*⟩ has not been defined, the package issues an `undefined-label` warning.

For example:

```
\mddefn{eq:ohm}{V}{The voltage across the resistor}
```

2.3 Textual concepts

\mdconcept `\mdconcept{⟨id⟩}{⟨text⟩}`

Marks an ordinary textual concept and records it for later output to the concept sidecar file. The command typesets ⟨*text*⟩ at the point where it appears in the document, and records the pair ⟨*id*⟩, ⟨*text*⟩.

Concept records are collected during the document and written to `\jobname.terms` at the end of document processing. Each distinct concept ID is written once, in first-use order.

If an already defined ⟨*id*⟩ is used with the same ⟨*text*⟩, the existing concept record is reused silently. If the ⟨*id*⟩ is used again with different text, the package issues an `inconsistent-concept` warning.

For example:

```
The \mdconcept{reference-oscillator}{reference oscillator}
provides the timing reference.
```

The rendered document contains the text `reference oscillator`. The concept sidecar file receives a record of the form:

```
reference-oscillator: reference oscillator
```

2.4 Semantic term macros

The `\mdterm` command generates mathematical terms according to well-defined semantic form. The command is a formatting macro. It does not write to a sidecar file.

\mdterm `\mdterm[⟨namespace⟩]{⟨symbol⟩}[⟨qualifier⟩]`
`(⟨association⟩)`

Composes a decorated mathematical term from a ⟨*symbol*⟩ and optional namespace, qualifier, and association components.

If ⟨*namespace*⟩ is supplied, it is rendered using `\nspace` (by default as a superscript). If ⟨*qualifier*⟩ is supplied, it is rendered using `\qual` (by default as a subscript). If the parenthesised ⟨*association*⟩ is supplied, it is rendered after using `\assoc` (by default in parentheses after the symbol).

For example:

```
\mdterm[SI]{V}[rms](input)
```

The package uses small mathematical formatting macros to control the typesetting of term notation. These macros may be redefined to change the format generated by `\mdterm`.

\nspace `\nspace{⟨text⟩}`

Typesets ⟨*text*⟩ as a namespace component (default is upright roman mathematics text).

For example:

```
\nspace{SI}
```

\assoc `\assoc{⟨text⟩}`

Typesets ⟨*text*⟩ as an association component (default is upright roman mathematics text).

For example:

```
\assoc{input}
```

\qual `\qual{⟨text⟩}`
`\qual{⟨text⟩}(⟨sub-qualifier⟩)`

Typesets ⟨*text*⟩ as a qualifier (default is upright roman mathematics text inside parentheses).

For example:

```
\qual{cal}
\qual{cal}(pre)
```

\tv `\tv{⟨text⟩}`

Typesets ⟨*text*⟩ as a parametric term (default in italic mathematics text).

For example:

```
\tv{offset}
```

3 Sidecar file formats

3.1 Model and term sidecar

The file `\jobname.models` is opened at the beginning of the document and closed at the end. Model records and mathematical-term definition records are written to this file.

A model record begins with the header line:

```
===== MODEL =====
```

It may then contain any of the following metadata lines, depending on which keys were supplied:

```
name: <name>
role: <role>
source: <source>
label: <label>
equation: <equation-body>
```

The `equation` line is always written for a model environment.

Mathematical-term records are written as simple blocks containing:

```
ref: <equation-label>
term: <term>
definition: <definition>
```

The current implementation does not add a separate term-record delimiter such as `--- TERM ---`.

3.2 Concept sidecar

The file `\jobname.terms` is written at the end of the document. It contains one block for each distinct concept ID recorded using `\mdconcept`. The output format is:

```
<id>: <text>
```

Blank lines are written between concept records.

4 Implementation

```
⟨*package⟩
\NeedsTeXFormat{LaTeX2e}
\ProvidesPackage{measuredtex}[2026/06/05 v0.1 Semantic measurement markup]
\RequirePackage { amsmath }
\ExplSyntaxOn
```

4.1 Messages

The package defines warning messages for undefined equation labels and for inconsistent reuse of textual concept identifiers.

```
\msg_new:nnn { measuredtex } { undefined-label }
  { md_term_write:~label~'#1'~is~not~defined. }
```

4.2 Metadata variables

These token-list variables hold the model metadata supplied through the optional key–value argument of the model environments.

```
\tl_new:N \l_md_name_tl
\tl_new:N \l_md_role_tl
\tl_new:N \l_md_source_tl
\tl_new:N \l_md_label_tl
\tl_new:N \l_md_model_tl
```

4.3 Output stream

Two output streams are declared. The model stream is opened at the beginning of the document and closed at the end. The terms stream is opened only when the collected textual concepts are written.

```
\iow_new:N \g_md_iow
\iow_new:N \g_md_terms_iow
\AtBeginDocument
  {
    \iow_open:Nn \g_md_iow { \jobname .models }
  }
\AtEndDocument
  {
    \iow_close:N \g_md_iow
    \md_concepts_write:
  }
```

4.4 Metadata keys

The `measuredtex/model` key family provides the supported model metadata fields. The `measuredtex/mddefn` family currently provides a `model` key, although the public term-definition commands presently take positional arguments.

```
\keys_define:nn { measuredtex / model }
  {
    name   .tl_set:N = \l_md_name_tl ,
    role   .tl_set:N = \l_md_role_tl ,
    source .tl_set:N = \l_md_source_tl ,
    label  .tl_set:N = \l_md_label_tl ,
  }
\keys_define:nn { measuredtex / mddefn }
  {
    model .tl_set:N = \l_md_model_tl ,
  }
```

4.5 Model-writing logic

`\md_model_write:nn` Clears all metadata variables, applies the supplied key–value pairs, and writes a model block to the sidecar file. Empty optional metadata fields are omitted. The equation body is written in string form.

```
\cs_new:Npn \md_model_write:nn #1 #2
  {
```

```
    \tl_clear:N \l_md_name_tl
    \tl_clear:N \l_md_role_tl
    \tl_clear:N \l_md_source_tl
    \tl_clear:N \l_md_label_tl
    \keys_set:nn { measuredtex / model } { #1 }
    \iow_now:Nn \g_md_iow { ===== ~ MODEL ~ ===== }
    \tl_if_empty:NF \l_md_name_tl
      {
        \iow_now:Nx \g_md_iow { name: ~ \tl_use:N \l_md_name_tl }
      }
    \tl_if_empty:NF \l_md_role_tl
      {
        \iow_now:Nx \g_md_iow { role: ~ \tl_use:N \l_md_role_tl }
      }
    \tl_if_empty:NF \l_md_source_tl
      {
        \iow_now:Nx \g_md_iow { source: ~ \tl_use:N \l_md_source_tl }
      }
    \tl_if_empty:NF \l_md_label_tl
      {
        \iow_now:Nx \g_md_iow { label: ~ \tl_use:N \l_md_label_tl }
      }
    \iow_now:Nx \g_md_iow { equation: ~ \tl_to_str:n {#2} }
    \iow_now:Nn \g_md_iow { }
  }
```

(*End of definition for* `\md_model_write:nn`.)

4.6 Document environments

mdeq

The `mdeq` environment records the model metadata and body, then typesets the body inside an `equation` environment. If a label was supplied through the metadata keys, it is applied inside the equation.

```
\NewDocumentEnvironment { mdeq } { O{} +b }
  {
    \md_model_write:nn {#1} {#2}
    \begin{equation}
      #2
      \tl_if_empty:NF \l_md_label_tl
        {
          \label { \tl_use:N \l_md_label_tl }
        }
    \end{equation}
  }
  { }
```

(*End of definition for* `mdeq`. *This function is documented on page* 2.)

mdalign

The `mdalign` environment is the corresponding wrapper for `align`.

```
\NewDocumentEnvironment { mdalign } { O{} +b }
  {
    \md_model_write:nn {#1} {#2}
    \begin{align}
      #2
```

```
      \tl_if_empty:NF \l_md_label_tl
        {
          \label { \tl_use:N \l_md_label_tl }
        }
    \end{align}
  }
  { }
```

(*End of definition for* `mdalign`. *This function is documented on page 3.*)

4.7 Term definition commands

`\md_term_write:nnn` The internal term-writing function checks that the referenced equation label exists and writes the reference, term, and definition fields to the model sidecar file.

```
\cs_new_protected:Npn \md_term_write:nnn #1 #2 #3
  {
    \cs_if_exist:cF { r@#1 }
      {
        \msg_warning:nnn { measuredtex } { undefined-label } {#1}
      }
    \iow_now:Nx  \g_md_iow { ref: ~ \tl_to_str:n {#1} }
    \iow_now:Nx  \g_md_iow { term: ~ \tl_to_str:n {#2} }
    \iow_now:Nx  \g_md_iow { definition: ~ \tl_to_str:n {#3} }
    \iow_now:Nn  \g_md_iow { }
  }
\NewDocumentCommand \mddefn { m m m }
  {
    \md_term_write:nnn {#1} {#2} {#3}
  }
\NewDocumentCommand \mddefntext { m m O{} m }
  {
    \md_term_write:nnn {#1} {#2} {#4}
    $#2$#3~#4
  }
\NewDocumentCommand \mddefntab { m m m }
  {
    \md_term_write:nnn {#1} {#2} {#3}
    $#2$ & #3
  }
\NewDocumentEnvironment { mddescription } { }
  {
    \begin{description}
  }
  {
    \end{description}
  }
\NewDocumentCommand \mditem { m m m }
  {
    \md_term_write:nnn {#1} {#2} {#3}
    \item[$#2$] #3
  }
\prop_new:N \g_md_concepts_prop
\seq_new:N  \g_md_concepts_seq
\msg_new:nnn { measuredtex } { duplicate-concept }
```

```
  { Concept~ID~'#1'~already~defined. }
\msg_new:nnn { measuredtex } { inconsistent-concept }
  {
    Concept~ID~'#1'~already~defined~as~'#2',~
    not~'#3'.
  }
\NewDocumentCommand \mdconcept { m m }
  {
    #2
    \prop_if_in:NnTF \g_md_concepts_prop {#1}
      {
        \prop_get:NnN \g_md_concepts_prop {#1} \l_tmpa_tl
        \tl_if_eq:NnF \l_tmpa_tl {#2}
          {
            \msg_warning:nnnnn
              { measuredtex }
              { inconsistent-concept }
              {#1}
              { \l_tmpa_tl }
              {#2}
          }
      }
      {
        \prop_put:Nnn \g_md_concepts_prop {#1} {#2}
        \seq_put_right:Nn \g_md_concepts_seq {#1}
      }
  }
% Write the collected concepts at the end
\cs_new_protected:Npn \md_concepts_write:
  {
    \iow_open:Nn \g_md_terms_iow { \jobname.terms }
    \seq_map_inline:Nn \g_md_concepts_seq
      {
        \prop_get:NnN \g_md_concepts_prop {##1} \l_tmpa_tl
        \iow_now:Nx \g_md_terms_iow
          { \tl_to_str:n {##1}: ~ \tl_to_str:V \l_tmpa_tl }
        \iow_now:Nn \g_md_terms_iow { }
      }
    \iow_close:N \g_md_terms_iow
  }
```

(*End of definition for* `\md_term_write:nnn`.)

4.8 Semantic term macros

These commands use conventional LaTeX 2$_\varepsilon$ syntax, so the implementation temporarily leaves `expl3` syntax.

```
\ExplSyntaxOff
```

`\nspace` Renders a namespace component in upright roman maths text.

```
\NewDocumentCommand{\nspace}{m}{%
  \mathrm{#1}%
}
```

(*End of definition for* `\nspace`. *This function is documented on page 6.*)

`\assoc` Renders an association component in upright roman maths text.

```
\NewDocumentCommand{\assoc}{m}{%
  \mathrm{#1}%
}
```

(*End of definition for* `\assoc`. *This function is documented on page 6.*)

`\tv` Renders a term value in italic maths text.

```
\NewDocumentCommand{\tv}{m}{%
  \mathit{#1}%
}
```

(*End of definition for* `\tv`. *This function is documented on page 6.*)

`\qual` Renders a qualifier, with an optional parenthesised sub-qualifier.

```
\NewDocumentCommand{\qual}{m d()}{%
  \mathrm{#1}\IfValueT{#2}{(\mathrm{#2})}%
}
```

(*End of definition for* `\qual`. *This function is documented on page 6.*)

`\mdterm` Composes a semantic mathematical term from an optional namespace, a required symbol, an optional qualifier, and an optional association.

```
\NewDocumentCommand{\mdterm}{o m o d()}{%
  \IfValueT{#1}%
    {{}^{\nspace{#1}}}%
  {#2}%
  \IfValueT{#3}%
    {_{\qual{#3}}}%
  \IfValueT{#4}%
    {(\assoc{#4})}%
}
```

(*End of definition for* `\mdterm`. *This function is documented on page 6.*)

```
\endinput
```

⟨/package⟩

Change History

v0.1

General: Initial public version. 1

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.